MeshTac: Mesh-Aligned Pseudo-Labeling for Label-Efficient Contact Geometry and Pose Estimation

Extended Version

Anonymous submission

***Abstract*— Known object meshes can provide geometric labels for vision-based tactile sensing, but registering each real contact to the mesh is costly. We propose MeshTac, a mesh-aligned pseudo-labeling method that learns registration from a small labeled set to label additional real tactile images. Given a teacher's initial contact estimate, a residual aligner iteratively corrects the contact state using the image and projected mesh geometry. The aligner learns from perturbed labeled contact states without additional real labels. The aligned mesh provides depth, normal, and pose targets for a student that predicts without mesh rendering. The aligner also supports mesh-assisted pose refinement at inference. On two GelSlim-type sensors, we compare MeshTac with real-only training, synthetic augmentation and self-training under matched real-label budgets. With 25 labeled contacts per object and no synthetic data, MeshTac reduces contact-depth error by 21% and 36%, matches models trained with twice as many labels, and improves pose estimation. Synthetic augmentation adds nothing within the training session but helps across capture sessions. A renderer trained on all real pairs appears to cut depth error by 24–26%, against 0–3% when restricted to the same 25 labels. Alignment thus improves supervision on real images, while simulation gains must be judged with the renderer's training labels included.**

I. Introduction

Contact geometry tells a robot where an object meets its fingertip and how the object is oriented. Vision-based tactile sensors such as GelSight, GelSlim and DIGIT [1]–[3] infer this geometry from images of a deforming elastomer. Their appearance varies with illumination, elastomer properties and camera response, so a predictor needs accurately labeled examples from the target sensor. We study known-object perception: object identity and a mesh are available, but registering every real contact to that mesh is costly.

Simulation reduces manual registration by generating tactile images and geometric labels from known meshes [4]–[6]. Learned renderers can narrow the gap to the target sensor, but fitting them also consumes real image–label pairs. If a renderer sees more real contacts than the downstream predictor, its synthetic data carries information outside the stated predictor budget. We therefore count renderer supervision as part of the real training-label budget.

Unlabeled tactile images already contain the target sensor's appearance. With a known mesh, labeling one of these images reduces to estimating its contact state: in-plane rotation, translation and indentation. A model trained on a small registered set can initialize this state, but even modest errors shift the projected contact and corrupt its dense targets. The mesh must therefore be aligned before it can supervise the image.

MeshTac learns this alignment as a residual correction (Fig. 1). Given a real image and the mesh depth at a candidate state, the aligner updates the state iteratively. Perturbing registered contacts supplies its training examples without further labeling. The corrected mesh then provides depth, normal and pose targets for the real image, allowing the student to learn from real appearance and predict without mesh rendering at standard inference.

We compare real-only training, synthetic augmentation, self-training and MeshTac under matched real-label budgets. Our contributions are:

- **Mesh-aligned pseudo-labeling.** The method learns contact-state corrections from a small registered set and converts unlabeled real images into depth, normal and pose supervision.
- **Controlled renderer study.** The apparent benefit of synthetic tactile augmentation depends strongly on whether the real image–label pairs used to train the renderer are counted in the supervision budget.
- **Evaluation across conditions.** Experiments on two sensors and a second capture session evaluate MeshTac's label efficiency and robustness; a separate held-out-object study characterizes when synthetic supervision helps, and the aligner is evaluated for pose refinement without retraining.

II. Related Work

Tactile simulation and transfer. Tactile simulators differ in how they model geometry and sensor appearance. TACTO [4] emphasizes optical rendering, Taxim [5] learns optical response and models marker motion from calibration examples, and Tacchi [6] simulates elastomer deformation. Physics-based [7], differentiable [8] and large-scale systems [9] make different tradeoffs between contact fidelity and computational cost. The remaining appearance gap can be addressed with learned image translation [10]–[12], including unpaired objectives [13]–[15], or with domain randomization [16]. These works primarily ask how to produce transferable synthetic observations. We instead ask how much real supervision the renderer consumes before those observations improve a downstream model. Sensor variation can also be addressed in feature space [17]–[19]; we hold that factor fixed with the same frozen SITR encoder [17] throughout.

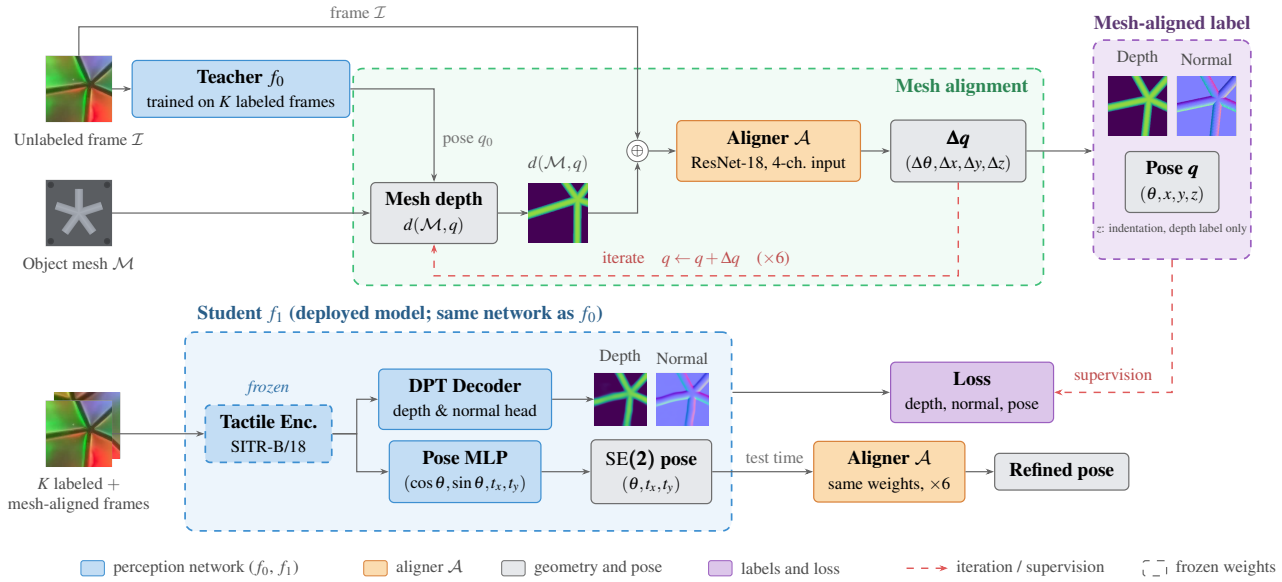


Fig. 1: **MeshTac overview.** Teacher f_0 initializes the contact state, and aligner $\mathcal{A}$ iteratively corrects it by comparing the tactile image with projected mesh depth. The aligned mesh provides depth, normal and planar-pose targets for student f_1. Every trainable component in the matched-budget pipeline shares the same K registered contacts; synthetic frames from a renderer are an optional addition evaluated separately. At standard inference, f_1 predicts without rendering the mesh. Optional pose refinement reuses $\mathcal{A}$ with the known mesh. The thumbnails show an actual $K = 25$ example and prediction.

Pseudo-labeling and pose refinement. Self-training extends supervision to unlabeled data with model predictions [20], [21], but teacher errors are inherited by the student [22]. Known geometry provides a way to correct those labels. DeepIM [23] learns iterative pose updates by comparing observed and rendered images, while Self6D [24] uses unlabeled real images for self-supervised pose learning. MeshTac uses refinement for a different purpose: it aligns a mesh to each tactile image and turns the corrected state into dense targets and a pose label. The resulting student predicts without mesh rendering, and label generation requires neither photorealistic rendering nor differentiation through a renderer.

III. Method: Mesh-Aligned Pseudo-Labeling

A. Problem Formulation

We study how to learn tactile depth, normals and pose when object meshes are known but registered real contacts are scarce. The comparison includes synthetic-data augmentation and mesh-aligned pseudo-labeling, with real training labels shared across components.

A tactile renderer maps a mesh $\mathcal{M}$ and contact state q (planar pose and indentation) to an image and dense labels, $(\hat{\mathcal{I}}, D, N) = \mathcal{R}_\theta(\mathcal{M}, q)$. The geometry stage $(M, q) \mapsto (D, N)$ is shared by every renderer we study; renderers differ only in the appearance operator $\mathcal{P}_\theta : (D, N) \mapsto \hat{\mathcal{I}}$ and in the real data $\mathcal{D}^r_{\text{cal}}$ used to fit θ. A downstream model f_ϕ predicts depth, normals and planar pose from a real image. We measure a renderer by the real test error of f_ϕ trained on its synthetic data plus a real subset $\mathcal{D}^r_{\text{task}}$, and we control the total real budget $\mathcal{D}^r_{\text{cal}} \cup \mathcal{D}^r_{\text{task}}$.

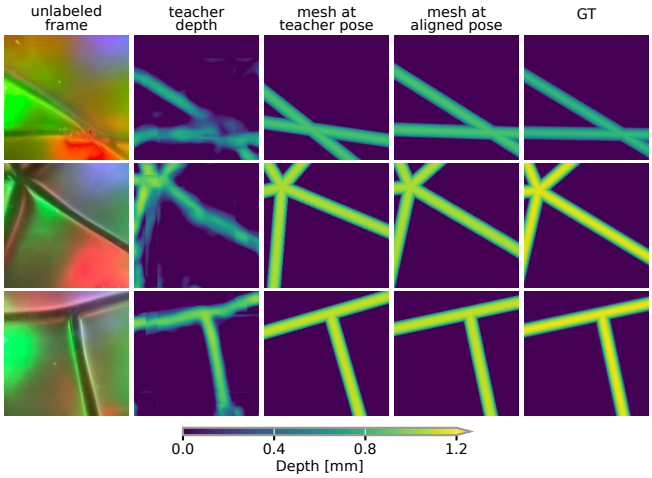


Fig. 2: **Pseudo-label construction.** Sensor A, $K = 25$. Columns compare teacher depth and mesh projections before and after alignment with GT. Each row is a median-IoU example from a subsampled candidate set for one object. All depth maps use the same scale (mm).

B. Mesh-Aligned Pseudo-Labels

MeshTac converts a small set of registered contacts and a larger pool of unlabeled real images into training data for a tactile predictor. The method first estimates each unlabeled contact state, then projects the known mesh to obtain mutually consistent depth, normal and pose targets.

Geometric labels. Let $d(\mathcal{M}, q)$ denote the depth of object mesh $\mathcal{M}$ at contact state $q = (\theta, x, y, z)$, where z is indentation. For a known object, these variables determine the mesh-derived depth map, its normals and the planar pose. Refinement therefore takes place in contact-state space, with the object geometry held fixed.

Learning the aligner. A ResNet-18, initialized from ImageNet, takes tactile RGB and candidate mesh depth as four input channels. It predicts a residual A in contact-state units:

$$\mathcal{A}\big(\mathcal{I}, d(\mathcal{M}, q^* + \delta)\big) \approx -\delta, \tag{1}$$

where q^* is the labeled state and δ is a sampled perturbation. The training set is produced by perturbing each registered contact, so learning the correction requires no labels beyond the original K contacts. We normalize the residual dimensions and optimize a smooth-L1 loss; Sec. IV gives the perturbation ranges and training details.

Labeling real images. For each unlabeled training image, teacher f_0 initializes planar pose and sets indentation to its maximum predicted depth. Given the known mesh, we perform six updates,

$$q_{t+1} = q_t + \mathcal{A}\big(\mathcal{I}, d(\mathcal{M}, q_t)\big), \tag{2}$$

clipping indentation to $[0.3, 1.6]$ mm. The final mesh depth, normals and planar pose label the image. We retain every pseudo-labeled training frame in the evaluated configuration (Sec. V-B).

Training the student. Student f_1 has the teacher's architecture and learns from the K labeled frames and the mesh-aligned frames. Synthetic frames from a renderer may be added as a separate variant; MeshTac itself uses none. Pseudo-labels are generated exclusively from the training split.

Optional test-time refinement. The student predicts from an image without mesh rendering. When the object identity and mesh are available, the same fixed aligner can refine its predicted pose using the test image. We evaluate this additional inference step separately in Sec. V-D.

IV. Experimental Setup

A. Sensors and Data

Sensor A. sensor A is a GelSlim-type fingertip sensor with a 17.5 mm square gel view imaged under six colored LED emitters. Images are center-cropped to 0.816 of the view and resized to 224×224.

Real data (session 1). A KUKA LBR Med arm presses the sensor onto 12 ridge-pattern objects of known geometry, recording the image and end-effector pose at every contact; ground-truth (GT) depth and normals are the mesh projected at the recorded pose (geometric targets, not membrane measurements). After removing frames whose image contact region disagrees with the GT, 4,669 frames remain (222–568 per object), one session per object, every tenth frame held out as validation (472 frames).

Real data (session 2). The same objects were re-placed and re-captured on another day (48 runs). GT poses were re-estimated per frame by maximizing the IoU between the image ridge mask and the projected mesh contact and manually reviewed; the curated test set has 854 frames of 11 objects (Sec. V-C). Nothing from this session is used for training or model selection.

Sensor B. GelSlim 4.0 [25] is the previous generation of the same fingertip family, with a marker array, a different illumination layout and a 12 mm field of view. Its public depth dataset [25] has the same 12 objects pressed by a parallel gripper at 1 mm indentation with mesh-derived depth and in-hand pose labels (135 test frames per object). We use the left finger, the central 12×12 mm, and report rotation after removing a per-object constant offset because its pose origin differs from ours.

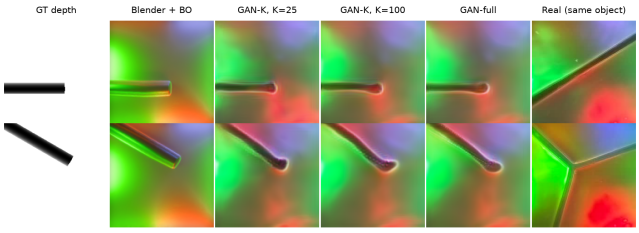


Fig. 3: **Two renderers, one contact.** Each row is one simulated contact (GT depth, left) rendered by the physically calibrated Blender renderer, by the learned renderer trained on $K = 25$ or $K = 100$ real pairs per object, and by the learned renderer trained on all real pairs (GAN-full). Right: a real frame of the same object. PBR reproduces the illumination field but not the dark contact shading of the real gel; the learned renderers reproduce the shading and inherit the illumination of their training frames.

The public depth labels are clipped at the nominal indentation and smoothed, so they are not exact mesh projections. For the MeshTac comparisons in Sec. V-C, a global search fits the planar mesh pose to each public label (0.018 mm mean residual). We canonicalize the fitted pose using the object's rotational symmetry and use its projected mesh depth for training and testing (*mesh-registered* labels). A separate set of renderer experiments in Sec. V-C retains the original public labels; those results are identified explicitly.

Contact sampler. For each object we sample contact cells over the mesh at 12 base rotations (30° apart), an indentation of 0.8–1.2 mm and in-plane jitter, and export D from the object z-buffer and N by finite differences. Simulated poses are restricted to each object's real rotation and translation ranges. The same 5,345 simulated contacts feed every renderer.

B. Physically Calibrated Renderer

The physically based renderer (PBR) is implemented in Blender/Cycles: a gel surface mesh is shrink-wrapped onto the object and smoothed (35 corrective iterations), shaded with a glossy/transparent material and a height-driven contact-darkening term, lit by six area emitters, and post-processed (saturation, brightness, contrast, haze, a low-frequency random gel warp). Emitter and post-processing parameters (16 scalars) are fitted by tree-based Bayesian optimization of an L_2 plus histogram distance between the real and rendered no-contact image (500 evaluations); gel and contact parameters were hand-tuned on a few contact frames. Thus, one no-contact image is the automatic calibration target, but additional contact images informed manual tuning. Their use must be distinguished from the counted paired training labels.

Domain-randomization baseline. To test whether careful calibration is necessary, we also render with randomized

sensor parameters: random HSV colors and strengths for each LED emitter, random gel optics, per session, with the geometry stage (scale, field of view, smoothness) fixed. This variant uses no image to fit its randomized appearance parameters; it retains the shared geometry configuration.

C. Learned Renderer

The learned renderer replaces $\mathcal{P}_\theta$ by a conditional GAN [26]: a five-level U-Net that takes the depth map and two coordinate channels and outputs an RGB image, a 70×70 spectral-normalized PatchGAN discriminator, and an adversarial $+ 100\,(\mathrm{L1} + \text{gradient L1} + \text{low-frequency L1})$ loss. Simulated depth is jittered by $\pm 15\%$ at generation time.

Real budget. GAN-K is trained on K real (depth, image) pairs per object, an evenly spaced subset of each object's training frames that is *the same* K frames the downstream model receives; GAN-LOO on all training frames of the 9 non-held-out objects; GAN-full on all 4,197 pairs. Small budgets train for at least 30k generator updates.

Variants. *Background-conditioned*: the session's no-contact image is appended as three input channels, providing session illumination from one background frame. *Physics-conditioned*: PBR's image of the same contact is appended, optionally predicting only a residual on top of it, optionally with the background. *Difference image*: the target is the image minus the session background. *Blender-pretrained*: pretraining on 30k (depth, Blender image) pairs, then fine-tuning on the real pairs.

Multi-sensor: the background-conditioned GAN is trained on 100 simulated sensor designs (random LED layouts and gel materials) plus optionally the target sensor's real pairs; this tests whether sensor diversity can replace sensor-specific real data. *Rotation coverage* changes the geometry stage instead: we vary how many of an object's real rotations appear in simulation (1, 2 or all 12) and, for the learned renderer, rotate the depth map before rendering so that every rotation is covered.

D. Zero-Label Alternative: Contact-Preserving Unpaired Refiner

We also test whether unlabeled real images can improve synthetic appearance without paired renderer supervision. Following unpaired image translation [11], [13], [15], we train a refiner to change the shading of a physical render while preserving its contact region. The refiner $\mathcal{F}$ takes the physical render $\hat{\mathcal{I}}_{\mathrm{phys}}$, the session background B and the contact mask m (simulated depth > 0, dilated by 4 px and feathered) and outputs

$$\hat{\mathcal{I}} = m \odot \mathcal{F}(\hat{\mathcal{I}}_{\mathrm{phys}}, B, m) + (1 - m) \odot B, \qquad (3)$$

The background is therefore taken from the real sensor, while the refiner changes only the contact region. Depth, normal and pose targets remain those of the physical render.

$\mathcal{F}$ is the same U-Net as the learned renderer with a 7-channel input. It is trained against a 70×70 PatchGAN on *unlabeled* real frames (image only; no depth, no pose) with an adversarial loss and a patchwise contrastive loss (PatchNCE [15]) between the encoder features of input and output, sampled preferentially inside m, to encourage preservation of the rendered ridge structure. We train for 10k generator updates with batch size 8 and Adam (2×10^{-4}, $\beta_1 = 0.5$).

Its real budget is the set of unlabeled frames plus one background frame per session, and the same leakage rule applies: in leave-object-out the held-out objects' frames are excluded, in K-shot the same K frames as the downstream model are used without their labels. With domain-randomized inputs, the refiner requires no appearance calibration or paired training labels; it retains the shared geometry configuration.

E. Downstream Tactile Model

The downstream model is identical for every renderer: a frozen sensor-invariant encoder (SITR-B/18 [17]) provides patch tokens at four depths; a DPT decoder [27] predicts depth and normals at 224×224; a pose head regresses $(\cos\theta, \sin\theta, t_x, t_y)$ from the class token and mean patch token. For the classification study, we add a 12-class head (LayerNorm on concatenated class and mean patch tokens, two-layer MLP with GELU) and train with cross-entropy alongside the other losses. The decoder, pose head and object embedding are trained (12M parameters). Losses are MSE on depth and normals, $1 - \cos(\theta - \hat{\theta})$ on rotation and L1 on translation, balanced by grouped uncertainty weighting [28].

When synthetic data is used, training mixes simulated and real frames. Simulated frames receive photometric augmentation and $\pm 15^\circ$ gel-spin rotation with matched labels; real frames are not augmented. We use AdamW with learning rate 2×10^{-4}, batch size 32 and a cosine schedule. Main sensor-A runs use 150 epochs, while sensor B and seed/ablation studies use 50. Depth and normal metrics use the checkpoint with the lowest validation depth MSE (at $K \leq 25$ within the first ten epochs). Pose metrics use the final epoch without selection; the pose head does not overfit (final-epoch and best-epoch rotation agree within 0.2°), whereas rotation at the early dense checkpoint mainly reflects its short training. The test-time refinement tables use the checkpoint with the lowest validation rotation error as the pose proposal. The legacy cross-session table reports pose from the depth checkpoint. In every case, checkpoint selection uses validation labels in addition to the K training labels.

F. MeshTac Implementation

Aligner perturbations span $\pm 8^\circ$, ± 1.5 mm in-plane and ± 0.15 mm in indentation; half use ranges scaled by 0.3. Training uses smooth L1 for 8,000 steps (batch 64, AdamW, learning rate 3×10^{-4}). Aligners train on real examples only, the teacher uses the K real frames, and the student receives no synthetic frame. The + *learned synth.* variant (sensor A) mixes 24 real and 40 synthetic examples generated by the K-pair renderer in each aligner batch, trains the teacher with learned-renderer data, and gives the student synthetic images from a renderer retrained on labeled and aligned real pairs.

On one RTX 6000 Ada GPU, aligner training takes about 20 minutes, labeling 0.5 s per frame (30 minutes for the

3,897 sensor-A frames), and student training 1.3 hours for 50 epochs or 2.2 hours for 150. The renderer, when used, adds about one hour.

G. Data-Access Protocol

Paired renderers and the aligner exclude validation and test images from training. The K-shot protocol additionally limits paired training labels to the same K frames per object. Validation labels remain available for model selection; the reported budget counts training labels. Physical-renderer manual tuning is described separately above.

Leave-object-out (LOO). Three objects (a two-line, a rod and a cross pattern; $1{,}244$ frames) are removed from all real training, excluded from paired-renderer and aligner training, and evaluated in full. Simulated contacts of the three meshes are available to every renderer ("new object with a CAD model but no real touches"). A *geometry control* additionally removes the three meshes from simulation.

K-shot. Every object contributes $K \in \{0, 10, 25, 50, 100\}$ evenly spaced real training frames. For $K > 0$, the learned renderer is trained on exactly those frames; the downstream model co-trains on them plus simulated data, or on them alone with the same number of updates. Evaluation is on the real validation split. The aligner of MeshTac receives the same K frames, and the remaining training frames may be used *without* their labels.

Cross-session and cross-sensor. Session-2 and sensor-B test frames are never used to train or select anything in the sensor-A study. In the sensor-B study the K-shot protocol is repeated with sensor-B real pairs and the sensor-B test split.

Learned renderer (full). GAN-full is trained on all $4{,}197$ sensor-A training pairs. In LOO, these include real pairs from held-out objects; in K-shot, they include training labels withheld from budget-matched methods, but no validation frames. We label this non-comparable reference "learned renderer (full)"; it is shown in italics and excluded from best-result highlighting.

Metrics. We report depth MSE over all pixels (mm^2), following prior work on this sensor. Because $\approx 90\%$ of pixels are background, we also report mean absolute error inside the GT contact region (contact MAE, mm) and contact IoU (predicted depth > 0.05 mm vs. GT > 0), which also penalizes false contact. Normals are evaluated by mean angular error inside the contact region (deg), and pose by mean rotation error (deg) and normalized translation L1.

V. Experiments

The experiments examine label efficiency, the contribution of alignment, and performance under changes in session or sensor. We then study renderer supervision and appearance variants to explain when synthetic data helps.

Sensor-A main runs use 150 epochs; seed checks, ablations and sensor-B runs use 50 (other exceptions are noted). Metrics are evaluated on real images; depth MSE in mm^2, contact MAE in mm, normals and rotation in degrees. Sections V-A–V-E evaluate MeshTac; Sections V-F–V-H report the renderer comparisons under explicit data-access constraints.

A. Label Efficiency

Tables I and II, and Fig. 4 compare, under the same training-label budget, real-only training, co-training with either renderer, self-training on the teacher's own predictions (Sec. V-B) and MeshTac. Each MeshTac run uses the $2{,}997$–$4{,}077$ training frames outside the K budget without their labels.

Simulation alone. Neither paired-renderer baseline improves both contact MAE and IoU for $K \leq 25$; the learned renderer helps from $K = 50$ (depth MSE -10%, -16% at $K = 100$), PBR by at most 5%. Both lower rotation error at $K = 10$ ($9.6^\circ \rightarrow 5$–6°) and raise it at $K = 100$.

MeshTac. Mesh-aligned supervision improves dense prediction at every evaluated budget: depth MSE by 19%, 22%, 40% and 38% and contact MAE by 11%, 21%, 30% and 29% at $K = 10$, 25, 50 and 100, with IoU $+0.04$ to $+0.07$ and normal error -3.3° to -5.7°. Over three seeds (50 epochs) the depth MSE is 0.0433 ± 0.0004 and 0.0222 ± 0.0004 at $K = 25$ and 100 against 0.0551 ± 0.0010 and 0.0366 ± 0.0008 for real-only training (Table III). Without checkpoint selection, at the final epoch, depth MSE is 0.0613, 0.0457, 0.0313 and 0.0239 against 0.0680, 0.0575, 0.0500 and 0.0396 (-10 to -40%).

The improvement also reduces the number of labels needed to reach a given depth error. MeshTac with 50 labels per object (0.0291) has lower depth MSE than real-only training with 100 (0.0369) and than the learned renderer at 100 (0.0311). At $K \geq 50$ it has lower error than the learned renderer (full) (depth MSE 0.0227 vs. 0.0274, contact MAE 0.166 vs. 0.205 at $K = 100$), and at $K = 25$ it has lower contact MAE (0.245 vs. 0.262) and equal IoU, despite 4% higher depth MSE.

Pose improves as well: MeshTac has the lowest budget-matched rotation error at every budget (4.7°, 2.8°, 2.7°, 2.5° against 9.8°, 5.6°, 4.3°, 3.2° for real-only training) and lower translation error for $K \leq 50$ (1.13, 0.91, 0.73 mm vs. 1.71, 1.13, 0.89; equal at $K = 100$). Test-time alignment reduces the errors further (Sec. V-D).

Adding synthetic frames from the K-pair renderer to the pipeline ("+ learned synth.": renderer data for aligner, teacher and student) does not improve the within-session results: depth MSE 0.0559, 0.0450, 0.0307, 0.0232 against 0.0521, 0.0433, 0.0291, 0.0227 without, rotation 3.7–4.0° against 2.5–4.7°, and IoU within ± 0.02. The renderer provides a benefit across sessions instead (Sec. V-C).

Seeds. Table III reports the comparison over three seeds. The depth-MSE differences between MeshTac and real-only training exceed the variation across these runs.

B. Ablation Study on Alignment Quality

We use the same unlabeled images to compare teacher predictions, render-and-compare search and learned alignment (Table IV). Evaluating both label quality and student performance connects the supervision changes to their downstream effect.

Self-training. With the teacher's own depth as label (stitched from five windows to cover the full frame, pose loss on the

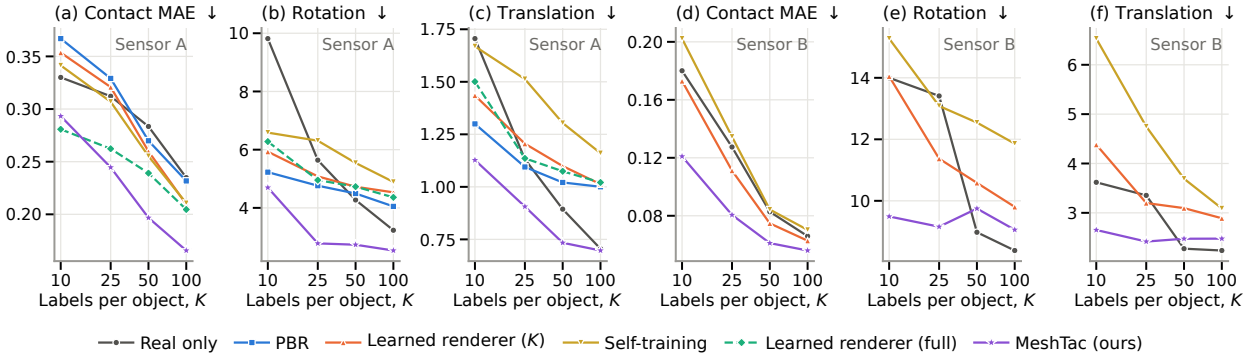


Fig. 4: **Label efficiency.** Contact MAE, rotation and translation error (final epoch) against the label budget. (a–c) Sensor-A validation results; (d–f) sensor-B test results. Lower is better. Lines connect evaluated budgets. Sensor A uses 150 epochs except the $K = 10$ learned renderer (full) (50); sensor B uses 50.

K	Method	Depth MSE↓	Contact MAE↓	IoU ↑	Normal (deg)↓	Rot. (deg)↓	Trans. (mm)↓
0	Blender only	**0.0793**	**0.408**	**0.572**	**48.6**	**8.6**	**1.73**
	GAN-full (full real set) only	*0.0459*	*0.274*	*0.705*	*44.6*	*7.2*	*1.64*
10	Real only	0.0642	0.330	0.625	29.8	9.8	1.71
	Blender	0.0636	0.367	0.602	30.8	5.2	1.30
	GAN-10	0.0624	0.354	0.597	30.6	5.9	1.44
	Teacher-label self-training	0.0574	0.341	0.592	28.3	6.6	1.67
	MeshTac, L1 search	0.0545	0.320	0.639	27.3	4.7	1.30
	MeshTac (ours)	**0.0521**	0.293	0.660	**26.5**	4.7	1.13
	+ learned synth.	0.0559	**0.288**	**0.682**	27.2	4.0	1.14
	synthetic frames for the aligner only	0.0571	0.293	0.678	27.3	**3.2**	**1.00**
	GAN-full (full real set)	*0.0484*	*0.281*	*0.693*	*28.5*	*6.3*	*1.50*
25	Real only	0.0558	0.312	0.664	28.2	5.6	1.13
	Blender	0.0549	0.329	0.656	27.5	4.8	1.09
	GAN-25	0.0544	0.321	0.626	28.3	5.1	1.21
	Blender ⊕ GAN-25	0.0554	0.324	0.667	27.4	4.8	1.12
	Refiner-25 (0 labels)	0.0551	0.297	0.672	27.1	5.7	1.26
	Refiner-U (0 labels, all unlabeled)	0.0566	0.309	0.666	27.8	5.8	1.27
	Self-trained GAN-25 (renderer only)	0.0538	0.326	0.641	28.0	4.9	1.14
	Teacher-label self-training, model only	0.0517	0.300	0.611	26.5	8.3	1.54
	center window only (no stitching)	0.0518	0.328	0.643	26.7	8.5	1.55
	Teacher-label self-training	0.0508	0.307	0.628	26.5	6.3	1.51
	round 2	0.0500	0.310	0.642	26.0	6.3	1.47
	MeshTac, L1 search	0.0482	0.291	0.655	25.5	4.5	1.15
	MeshTac (ours)	**0.0433**	**0.245**	0.726	**23.3**	**2.8**	0.91
	+ learned synth.	0.0450	0.249	**0.727**	24.0	3.7	1.00
	synthetic frames for the aligner only	0.0456	0.259	0.715	24.7	2.8	**0.86**
	GAN-full (full real set)	*0.0417*	*0.262*	*0.724*	*25.5*	*5.0*	*1.14*

TABLE I: **K-shot with a shared real budget.** Every object contributes K real frames; GAN-K is trained on the same frames. MeshTac and self-trained rows additionally use the unlabeled training frames (Sec. V-B); refiner training uses no paired label. Real validation split (472 frames). Learned renderer (full) rows use the renderer trained on all real pairs (the $K = 10$ learned renderer (full) row is a 50-epoch run). + *learned synth.*: MeshTac with the K-pair renderer added to aligner, teacher and student training.

labeled frames only), depth MSE improves by 8% but contact MAE and IoU do not: the pseudo labels inherit the teacher's localization error (label IoU 0.56), indicating that a lower all-pixel error need not imply improved localization. Adding the teacher's pose as a label improves rotation accuracy with little change in dense metrics, and a second round of self-training changes depth MSE by at most 4% in either direction.

L1 render-and-compare. Replacing the aligner by a search—fit the mesh to the teacher depth on a pose grid, then pick, among 567 neighbouring poses, the one whose GAN-K render is closest to the frame in image L1—helps less (label IoU 0.63 with the half-frame gate, pose error 2.5°) and requires gating: on the half of the frames with the larger render error the mesh label is no better than the teacher's. Image similarity does not reliably identify the correct pose with this few-shot renderer; at $K = 25$ its render at the true pose is farther from the frame than its render at the selected pose on 93% of the frames. This measures score ordering, rather than a pose-failure rate.

Learned aligner. The aligner lowers the teacher's pose error

K	Method	Depth MSE↓	Contact MAE↓	IoU ↑	Normal (deg)↓	Rot. (deg)↓	Trans. (mm)↓
50	Real only	0.0482	0.283	0.690	25.5	4.3	0.89
	Blender	0.0478	0.270	0.701	25.2	4.5	1.02
	GAN-50	0.0434	0.260	0.716	24.3	4.7	1.10
	Teacher-label self-training	0.0381	0.255	0.706	22.7	5.5	1.30
	MeshTac, L1 search	0.0384	0.247	0.714	22.7	4.2	1.03
	MeshTac (ours)	**0.0291**	**0.197**	0.761	**19.8**	**2.7**	**0.73**
	+ learned synth.	0.0307	0.206	**0.769**	20.4	3.7	0.95
	synthetic frames for the aligner only	0.0332	0.215	0.764	21.1	3.0	0.76
	GAN-full (full real set)	*0.0363*	*0.239*	*0.747*	*22.7*	*4.7*	*1.07*
100	Real only	0.0369	0.235	0.744	22.1	3.2	0.71
	Blender	0.0350	0.232	0.727	22.0	4.1	1.00
	GAN-100	0.0311	0.210	0.773	20.8	4.5	1.01
	Blender ⊕ GAN-100	0.0329	0.213	0.760	20.5	4.2	0.99
	Refiner-100 (0 labels)	0.0340	0.224	0.761	21.1	4.9	1.15
	Refiner-U (0 labels, all unlabeled)	0.0329	0.228	0.747	21.1	4.9	1.16
	Self-trained GAN-100 (renderer only)	0.0308	0.207	0.774	20.6	4.4	1.01
	Teacher-label self-training, model only	0.0290	0.221	0.744	19.8	4.5	1.09
	Teacher-label self-training	0.0285	0.211	0.750	19.7	4.9	1.16
	round 2	0.0295	0.217	0.736	20.2	4.9	1.16
	MeshTac, L1 search	0.0276	0.203	0.771	19.4	4.1	0.96
	MeshTac (ours)	**0.0227**	**0.166**	**0.818**	**16.9**	**2.5**	**0.70**
	+ learned synth.	0.0232	0.168	0.803	17.1	3.7	0.90
	synthetic frames for the aligner only	0.0237	0.168	0.809	17.1	2.7	0.71
	GAN-full (full real set)	*0.0274*	*0.205*	*0.777*	*19.9*	*4.4*	*1.02*

TABLE II: **Larger training-label budgets.** Continuation of Table I, $K = 50$ and 100. All data-access and metric conventions are unchanged.

K	Method	Depth MSE	Cont. MAE	IoU	Rot.
25	Real only	0.0551 ± 0.0010	0.314 ± 0.010	0.665 ± 0.004	5.6 ± 0.0
25	GAN-25	0.0536 ± 0.0013	0.314 ± 0.008	0.655 ± 0.032	6.1 ± 0.0
25	Teacher-label self-training	0.0507 ± 0.0001	0.310 ± 0.004	0.624 ± 0.002	7.5 ± 0.1
25	MeshTac, L1 search	0.0482 ± 0.0004	0.289 ± 0.004	0.648 ± 0.010	5.5 ± 0.0
25	MeshTac	0.0433 ± 0.0004	0.247 ± 0.000	0.722 ± 0.000	4.0 ± 0.0
25	+ learned synth.	0.0447 ± 0.0009	0.246 ± 0.002	0.729 ± 0.002	4.9 ± 0.1
100	Real only	0.0366 ± 0.0008	0.234 ± 0.006	0.750 ± 0.004	3.9 ± 0.0
100	GAN-100	0.0299 ± 0.0004	0.201 ± 0.002	0.779 ± 0.001	5.5 ± 0.0
100	Teacher-label self-training	0.0280 ± 0.0001	0.209 ± 0.002	0.743 ± 0.007	6.1 ± 0.1
100	MeshTac, L1 search	0.0275 ± 0.0002	0.201 ± 0.003	0.767 ± 0.005	5.0 ± 0.0
100	MeshTac	0.0222 ± 0.0004	0.163 ± 0.002	0.821 ± 0.002	3.8 ± 0.1
100	+ learned synth.	0.0229 ± 0.0003	0.164 ± 0.000	0.818 ± 0.000	4.8 ± 0.0

TABLE III: **Seeds.** Mean ± standard deviation over three seeds (50 epochs) on the real validation split. "MeshTac": real-only teacher, no synthetic frame; "+ learned synth.": the K-pair renderer added to aligner, teacher and student.

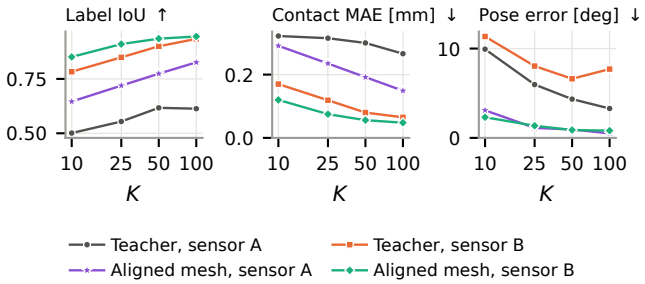


Fig. 5: **Pseudo-label quality.** Teacher predictions and aligned mesh labels, evaluated against withheld training GT on each sensor. The withheld GT is not used for learning.

from 6.0° to 1.1° on the 3,897 unlabeled frames and yields labels with IoU 0.72 and contact MAE 0.24 mm (teacher: 0.55, 0.32). Label quality grows with the budget (IoU 0.65, 0.72, 0.77, 0.83 and pose error 3.1°, 1.1°, 0.9°, 0.5° for K = 10–100; teacher 9.9°, 6.0°, 4.3°, 3.3°). Relative to teacher-depth self-training, the student lowers contact error (0.310 → 0.247) and raises IoU (0.624 → 0.722). Without any refinement, the mesh projected at the teacher's pose is a worse label than the teacher's own depth (IoU 0.47 vs. 0.56, contact MAE 0.40 vs. 0.32 mm) and the student trained on it falls below real-only training (depth MSE 0.0641, contact MAE 0.384, IoU 0.497): exact geometry placed at a wrong pose misleads the student, so the alignment step is what makes mesh supervision usable. Fig. 6 shows the convergence: from the teacher's pose the mean error falls to 2.1° after one update, 1.1° after three and 1.05° after five, then drifts up slightly (1.2° at ten); six updates recover perturbations of up to 15° to 0.6°, beyond the ±8° training range, and degrade only from 20° (0.8°; 1.5° at 25°, 3.2° at 30°).

Aligner training. Training the aligner on real examples alone

Labels of U		Pseudo-label quality			Downstream			
		IoU	Cont.	Rot.	Depth	Cont.	IoU	Rot.
Depth	Pose	↑	MAE↓	(deg)↓	MSE↓	MAE↓	↑	(deg)↓
teacher	–	0.56	0.32	–	0.0507	0.310	0.624	7.5
teacher	teacher	0.56	0.32	4.9	0.0508	0.310	0.622	6.0
mesh	teacher pose	0.47	0.40	6.0	0.0641	0.384	0.497	5.7
mesh$^{1/2}$	L1$^{1/2}$	0.63	0.30	1.4	0.0485	0.286	0.639	6.5
mesh	L1	0.64	0.30	2.5	0.0489	0.292	0.657	5.5
mesh$^{1/2}$	L1	0.63	0.30	2.5	0.0482	0.289	0.648	5.5
mesh	aligner‡	0.71	0.24	1.1	0.0447	0.248	0.730	5.0
mesh	aligner†	0.71	0.24	1.1	0.0447	0.246	0.729	4.9
mesh	aligner, no renderer	0.72	0.23	1.1	0.0433	0.247	0.722	4.0

TABLE IV: **Alignment ablation** ($K = 25$, 50 epochs; first, fifth and last row are three-seed means). Left: quality of the pseudo labels of the unlabeled frames against their withheld GT labels (evaluation only). Right: real validation error of the model trained on them. Depth label: teacher prediction or mesh depth at the fitted pose ($^{1/2}$: mesh on the half of each object's frames with the lowest render error, teacher on the rest). Pose label: none (pose loss on the K labeled frames only), teacher pose, pose found by L1 render-and-compare search with GAN-K ($^{1/2}$: on the confident half only), or the learned aligner. ‡: synthetic data from the teacher-pair renderer; †: MeshTac (renderer retrained on the aligned pairs).

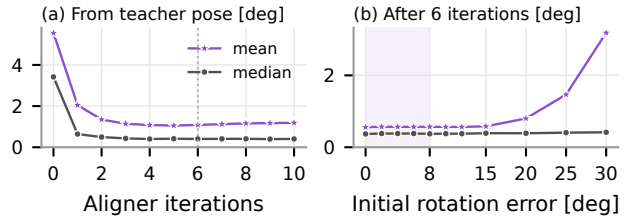


Fig. 6: **Aligner convergence** (sensor A, $K = 25$, 472 validation frames). (a) Rotation error against the number of aligner updates, starting from the teacher's predicted pose; the dotted line marks the six updates used throughout. (b) Error after six updates when the GT pose is perturbed by a rotation of the given magnitude plus a random in-plane shift of up to 1.5 mm; the shaded band is the $\pm 8°$ training range.

gives label IoU 0.72; adding synthetic examples gives 0.71, and synthetic-only training is less accurate (0.56). Label IoU changes little across the tested settings (half or double the perturbation range: 0.72, 0.73; 2k or 20k steps: 0.73, 0.72; ResNet-34 or -50 backbone: 0.73, 0.71), while dropping the ImageNet initialization costs accuracy (0.67).

Synthetic student data. Student depth MSE is unchanged whether its synthetic frames come from the renderer retrained on aligned pairs or on teacher-labeled pairs (depth MSE 0.0447 in both cases), and within-session depth gains also persist without synthetic data (last row: 0.0433 ± 0.0004 over three seeds with a real-only teacher and no synthetic frame, against 0.0447 ± 0.0009 with the renderer).

Unlabeled data and filtering. Alignment improves labels even on the frames rejected by the render-error gate (IoU 0.66 vs. 0.55 for the teacher), supporting their inclusion in training. A quarter of the unlabeled frames already gives most of the depth-MSE gain (depth MSE 0.0453, 0.0452 and 0.0447 with 25%, 50% and 100% of U), while IoU keeps growing with their number (0.710, 0.725, 0.729).

Repeated alignment. A second round, with the student as the new teacher, offers little further improvement: although the student proposes poses 1.5° closer than the original teacher, the aligned labels (IoU 0.65, 0.71 at $K = 10$, 25) and the retrained student (depth MSE 0.0570, 0.0445; IoU 0.674, 0.717) are within seed variation of the first round.

An unpaired sim-to-real refiner of the physical render, which uses the unlabeled frames without any pose, does not improve on its input (contact MAE 0.349 vs. 0.346 on unseen objects).

Self-trained renderer. Following self-training [20], [21], the K-shot downstream model pseudo-labels U (depth predicted on five windows and stitched back to the full frame; frames kept when the predicted contact overlaps the image's black-hat ridge mask, IoU ≥ 0.2: 3,260 frames at $K = 25$, 2,677 at $K = 100$), and GAN-K is retrained on the K true plus the pseudo pairs. The pseudo labels are as accurate as the labeler (0.32 and 0.24 mm contact MAE against the withheld GT labels), and the renderer's fidelity improves by 5% at both budgets ($0.108 \to 0.102$, $0.077 \to 0.073$), while downstream depth MSE improves by about 1%: depth MSE $0.0544 \to 0.0538$ at $K = 25$ and $0.0311 \to 0.0308$ at $K = 100$ (contact MAE 0.207, lower than the paired-renderer baseline at $K = 100$). With teacher labels the renderer thus gains little from more real images.

Aligned pairs improve renderer fidelity further: contact-region error falls by 13–19% at every budget ($0.121 \to 0.105$ at $K = 10$, $0.108 \to 0.090$ at 25, $0.095 \to 0.077$ at 50, $0.077 \to 0.065$ at 100; all-pairs 0.057), the lowest contact-region errors among the budget-matched renderers. Replacing the teacher-pair renderer with this aligned-pair renderer nevertheless leaves the student's depth MSE unchanged (Table IV). In this within-session comparison, improved synthetic appearance adds little once the real frames carry aligned labels.

Window stitching. Without alignment the label is the teacher depth, and labeling U on the center window alone leaves the pseudo depth empty outside the 0.816 crop (pseudo-label contact MAE 0.41 mm over the full frame instead of 0.32). The self-trained model then has the same depth MSE (0.0518 vs. 0.0517, model only, $K = 25$; Table I) but a worse contact MAE (0.328 vs. 0.300) and a 30% larger peak-depth error, and the gap persists across sessions (contact MAE 0.358 vs. 0.318): the rotation augmentation brings the frame border into the training crop, where unstitched labels are wrong.

Rounds. Without alignment, relabeling U with f_1 and repeating the renderer and model steps improves the pseudo labels at $K = 25$ (label IoU $0.56 \to 0.60$, contact MAE $0.32 \to 0.31$ mm, 3,516 frames kept instead of 3,260) but not at $K = 100$ ($0.66 \to 0.65$), and the downstream change is small in both directions: depth MSE $0.0508 \to 0.0500$ and IoU $0.628 \to 0.642$ at $K = 25$, $0.0285 \to 0.0295$ and $0.750 \to 0.736$ at $K = 100$, rotation unchanged (6.2°, 4.9°). Additional self-training rounds have limited benefit in these runs. A second MeshTac round likewise yields little further improvement, as reported above.

Selection of the labeled frames. Table VI repeats the MeshTac pipeline with two random selections of the 25 labeled frames per object. Label quality (IoU 0.72, 0.71; pose

Method	Contact MAE↓	IoU ↑	Normal (deg)↓	Rot. (deg)↓	Rot.* (deg)↓
Real only (9 objects, session 1)	0.315	0.675	25.4	16.4	6.5
Blender	0.294	0.701	24.9	11.5	5.1
Blender (domain rand.)	0.294	0.691	25.5	12.3	5.9
Blender, bg-norm.	0.291	0.698	24.9	**9.7**	**4.7**
GAN-LOO	0.272	0.723	24.1	11.1	5.2
Bg-cond. GAN-LOO	0.270	0.727	24.2	10.5	4.7
Blender ⊕ GAN-LOO	0.275	0.724	24.0	9.8	4.8
Refiner (0 labels)	0.290	0.714	24.6	12.0	5.3
GAN-full (full real set)	*0.259*	*0.748*	*22.9*	*10.1*	*4.8*
Blender only ($K = 0$)	0.425	0.561	50.7	25.2	5.6
$K = 25$ real only	0.368	0.598	30.7	20.8	5.4
$K = 25$ + Blender	0.382	0.615	30.1	23.0	5.4
$K = 25$ + GAN-25	0.342	0.589	28.7	25.7	5.3
$K = 25$ + refiner-25 (0 labels)	0.338	0.635	29.5	15.5	5.6
$K = 25$ + self-trained GAN-25	0.363	0.606	29.1	20.4	5.5
$K = 25$ + teacher-label self-training	0.335	0.598	27.5	13.1	5.5
$K = 25$ + MeshTac, L1 search	0.322	0.621	27.4	17.2	5.1
$K = 25$ + MeshTac	0.306	0.654	27.3	17.3	5.2
$K = 25$ + MeshTac + learned synth.	0.310	0.660	27.2	13.8	5.3
$K = 100$ real only	0.351	0.657	27.5	17.5	5.5
$K = 100$ + Blender	0.299	0.647	25.3	16.4	5.1
$K = 100$ + GAN-100	0.295	0.713	25.6	13.7	5.4
$K = 100$ + refiner-100 (0 labels)	0.330	0.688	26.2	16.9	6.1
$K = 100$ + self-trained GAN-100	0.310	0.715	26.1	12.1	4.9
$K = 100$ + teacher-label self-training	0.300	0.707	24.2	11.0	4.9
$K = 100$ + MeshTac, L1 search	0.293	0.718	23.5	12.5	5.0
$K = 100$ + MeshTac	0.288	0.707	24.6	15.1	4.7
$K = 100$ + MeshTac + learned synth.	**0.252**	**0.743**	**22.8**	14.4	5.2

TABLE V: **Cross-session.** Sensor-A models evaluated on the curated session-2 set (854 frames, 11 objects; poses re-estimated per frame and manually reviewed). Rot.: raw; Rot.*: after removing a per-object constant offset (the re-placement of the object). Rows without a K use the session-1 training set of nine objects; object coverage therefore differs from the K-shot rows. Pose metrics use the depth checkpoint.

Training	Depth MSE↓	Contact MAE↓	IoU ↑
Real only, selection 0	0.0558	0.303	0.666
Real only, selection 1	0.0578	0.314	0.655
Real only, selection 2	0.0585	0.324	0.650
MeshTac, selection 0	0.0435	0.247	0.722
MeshTac, selection 1	0.0426	0.252	0.724
MeshTac, selection 2	0.0425	0.244	0.718

TABLE VI: **Robustness to the choice of the K labeled frames** (sensor A, $K = 25$, 50 epochs, real validation split). Selection 0 is the evenly spaced selection used elsewhere; selections 1 and 2 are random permutations of the training frames with the validation split unchanged. Every component (teacher, aligner, student) is retrained on the new selection.

error 1.8°, 1.6°) and student error are within seed variation of the evenly spaced selection, while real-only training degrades slightly on the random selections.

Where alignment fails. Against the withheld GT of the 3,897 unlabeled frames ($K = 25$), the aligned pose has a mean rotation error of 1.07° and a median of 0.47°; 3.6% of the frames exceed 2°, 1.6% exceed 5° and 1.2% exceed 10° (teacher: 65%, 35%, 16%). The failures concentrate on the five-line object pattern_06 (16.5% of its frames above 5°; every other object below 4%), whose near-symmetric ridge layout admits a second plausible alignment, and the worst 5% of frames carry 53% of the summed rotation error (mean 0.53° without them). Translation error (mean 0.98 mm, median 0.52) is dominated by shifts along the ridge direction: 73% of the rod frames and 20–40% of the frames of the line patterns exceed 1 mm, against 2–6% for the objects with crossing ridges (pattern_32, 33, 35, 36). Such shifts leave the depth label unchanged, which is why the label IoU is high despite them. At $K = 100$ the mean rotation error is 0.68° and 1.3% of the frames exceed 5°.

C. Generalization Across Sessions and Sensors

Second session. The second session tests changes in appearance and object placement without retraining (Table V). MeshTac reduces contact MAE relative to the budget-matched real-only models by 17% at $K = 25$ and 18% at $K = 100$, with MAE/IoU of 0.306/0.654 and 0.288/0.707; with synthetic frames the values are 0.310/0.660 and 0.252/0.743 (−28% at $K = 100$). The nine-object real-only baseline (0.315, 0.675) has different object coverage and serves as context rather than a matched label-efficiency comparison.

Synthetic data contributes more here than within the training session. At $K = 100$, adding the renderer lowers MeshTac's contact MAE from 0.288 to 0.252 and raises IoU from 0.707 to 0.743; at $K = 25$ it changes neither (0.306 vs. 0.310). MeshTac alone still improves on self-training in MAE (0.300 at $K = 100$) and matches its IoU. Together with the within-session ablation, this comparison supports synthetic data as a useful source of robustness under the evaluated session change.

Renderer baselines across sessions. Table V evaluates the sensor-A models on the second capture. Dense error is 1.5–3× the same-session value (e.g. contact MAE 0.315 mm against 0.117 for the model trained on all session-1 training data from nine objects), although the use of re-estimated GT poses also affects the comparison between sessions. The renderer baselines improve contact reconstruction and pose for the nine-object models. With the nine-object session-1 real set, contact MAE falls from 0.315 mm (real only) to 0.29 with the physical and 0.27 with the learned renderer, IoU rises from 0.675 to 0.69–0.70 and 0.72–0.73, normal error falls from 25.4° to 24.0–24.9° with the calibrated and learned renderers, and raw rotation error drops from 16.4° to 9.7–11.5°; the geometry control (28.7°) shows that the pose gain again comes from mesh coverage.

The domain-randomized renderer matches the calibrated one on dense error (0.294, IoU 0.691) and is slightly behind

on pose (12.3°), consistent with the pose trend on unseen objects. About $2/3$ of the remaining rotation error is a per-object constant (Rot.* 4.7–6.5° for every model), consistent with an offset introduced by object re-placement.

In the K-shot regime the dense gain grows with K: at $K = 25$ only the learned renderer lowers contact MAE ($0.368 \rightarrow 0.342$), at $K = 100$ both renderers do ($0.351 \rightarrow 0.295$–0.299) and the learned one also raises IoU ($0.657 \rightarrow 0.713$) and lowers rotation error ($17.5° \rightarrow 13.7°$), whereas simulation-only training does not transfer dense geometry across sessions (IoU 0.56, normal 51°): these results distinguish the benefit of synthetic pose coverage from the difficulty of transferring dense predictions across appearances.

Second sensor. MeshTac. On GelSlim 4.0 (1,620 test frames) we retrain MeshTac with the same aligner settings (Table VII, Fig. 4d–f). The public depth labels of this dataset are not mesh projections, so we re-registered them: the mesh pose that best explains each label was fitted (0.018 mm mean residual) and canonicalized against the object's rotational symmetry, and every method was retrained and tested on these mesh-registered labels.

The few-shot teacher is already more accurate on this sensor (label IoU 0.78, 0.85, 0.90, 0.93 on the unlabeled frames at $K = 10$, 25, 50, 100; 0.55 at $K = 25$ on sensor A), yet the aligner improves on it at every budget (IoU 0.85, 0.91, 0.94, 0.95; pose error 2.3°, 1.4°, 0.9°, 0.8° against 6.6–11.4°).

At $K = 25$ MeshTac lowers depth MSE from 0.0179 ± 0.0004 to 0.0076 ± 0.0001 and contact MAE from 0.127 ± 0.001 to 0.082 ± 0.002 mm over real-only training (three seeds) and raises IoU from 0.849 to 0.912; with synthetic frames the values are 0.0075 ± 0.0001, 0.078 ± 0.0003 and 0.903. The gains exceed those of the compared baselines: the learned renderer trained on the 25 pairs reaches 0.111 mm

K	Method	Depth MSE↓	Contact MAE↓	IoU ↑	Normal (deg)↓	Rot. (deg)↓
10	Real only	0.0314	0.180	0.780	22.5	14.0
	Learned renderer	0.0290	0.173	0.801	21.7	14.0
	Self-training	0.0301	0.202	0.773	22.5	15.3
	MeshTac (ours)	**0.0170**	**0.121**	**0.856**	**19.7**	**9.5**
25	Real only	0.0182	0.127	0.847	19.6	13.4
	Learned renderer	0.0148	0.111	0.877	18.3	11.4
	Self-training	0.0175	0.135	0.846	19.7	13.1
	MeshTac (ours)	0.0076	0.081	**0.911**	16.3	**9.2**
	+ learned synth.	**0.0075**	**0.078**	0.904	**16.2**	9.9
50	Real only	0.0088	0.083	0.899	16.9	**9.0**
	Learned renderer	0.0070	0.075	0.910	16.0	10.6
	Self-training	0.0081	0.084	0.901	16.4	12.5
	MeshTac (ours)	**0.0041**	**0.061**	**0.933**	**14.8**	9.7
100	Real only	0.0051	0.066	0.932	15.1	**8.4**
	Learned renderer	0.0052	0.063	0.928	15.0	9.8
	Self-training	0.0047	0.070	0.933	14.9	11.9
	MeshTac (ours)	**0.0034**	**0.056**	**0.944**	**14.4**	9.1

TABLE VII: **Second sensor (GelSlim 4.0) with mesh-registered labels**; 1,620 test frames, 50 epochs. "+ learned synth.": learned renderer trained on the K pairs added to aligner, teacher and student. Self-training uses the teacher's own depth as label.

(IoU 0.877) and self-training on the teacher's depth does not improve on real data (0.135 mm, 0.846).

Across budgets, MeshTac lowers contact MAE from 0.180, 0.127, 0.083, 0.066 to 0.121, 0.081, 0.061, 0.056 mm at $K =$ 10–100 (-15 to -36%) and depth MSE by 33–58%. At $K = 25$ and 50, its contact MAE is no higher than real-only training with twice as many labels. Rotation error improves at 10 and 25 labels ($14.0° \rightarrow 9.5°$, $13.4° \rightarrow 9.2°$) but not beyond (9° either way). Scored against the original public labels, MeshTac keeps its advantage (contact MAE $0.137 \rightarrow 0.098$ mm at $K = 25$).

The baselines follow the sensor-A pattern (Table VII, Fig. 4d–f). The learned renderer trained on the K pairs lowers contact MAE by 4%, 13%, 10% and 5% at $K = 10$, 25, 50 and 100 (0.173, 0.111, 0.075, 0.063 mm) and raises IoU by up to 0.03 for $K \leq 50$, while teacher-depth self-training does not improve on real-only training at any budget (0.202, 0.135, 0.084, 0.070 mm). MeshTac lowers contact MAE by 33%, 36%, 26% and 15% at the same budgets; its advantage over the learned renderer is largest at $K \leq 25$ (0.121 vs. 0.173 and 0.081 vs. 0.111 mm) and remains at $K = 100$ (0.056 vs. 0.063). Neither baseline improves rotation at $K \geq 50$.

Seeds. Table IX repeats the real-only and MeshTac comparison with three seeds at every budget; the standard deviations are at least an order of magnitude below the differences between the two.

Renderer baselines (original labels). Sensor B has a marker array, a different LED layout and a smaller field of view (Table VIII). *Zero-shot*, sensor-A models recover contact only partially (IoU 0.28 for the all-real model, 0.37 for the simulation-only model: the model with more sensor-A real supervision transfers less well in this comparison) and rotation not at all (residual $\approx 11.5°$ for every model, the value a constant predictor attains on the $\pm 22.5°$ label range).

With sensor-B real pairs, the K-shot picture of Sec. V-A reappears: a learned renderer trained on the sensor's own $K = 25$ pairs lowers contact MAE from 0.139 to 0.100 mm (-29%), normal error from 17.2° to 14.6° and raises IoU from 0.82 to 0.85; at $K = 10$ the gain is 6%, at $K = 100$, synthetic data brings little additional dense improvement. The sensor-A PBR, calibrated to a different sensor, still helps at $K = 25$ (0.125) but less; the domain-randomized renderer matches it (0.126) without using any real data from either sensor. At $K = 100$ real data alone is already at 0.058 mm and simulation neither helps nor hurts dense error; the domain-randomized renderer (0.062) is close. Rotation is not improved by either renderer and is degraded at $K = 100$ ($7.5° \rightarrow 9$–$10°$), as on sensor A. The transition is earlier on sensor B ($K \approx 25$ vs. 50), plausibly because its uniform 1 mm indentation and larger contact regions make the depth-to-image mapping easier to learn from few pairs.

Self-training (original labels). With the original labels and the GAN-25 model as labeler, teacher-label self-training lowers depth MSE from 0.0152 (real only) to 0.0098 and contact MAE from 0.140 to 0.100 mm at $K = 25$ without synthetic data, i.e. it approaches the labeler's depth MSE (0.0107) and matches its contact MAE (0.100); with the

K	Method	Depth MSE↓	Contact MAE↓	IoU ↑	Normal (deg)↓	Rot. (deg)↓
0	Sensor-A model, real only	0.1486	0.513	0.280	25.7	11.6
	Sensor-A model, Blender only	0.1175	0.413	0.370	34.3	11.6
10	Real only	0.0248	0.165	0.803	20.5	11.1
	Blender (sensor-A calib.)	0.0266	0.168	0.803	20.0	11.5
	GAN-10 (sensor B)	0.0237	0.155	0.809	19.0	11.3
25	Real only	0.0152	0.139	0.824	17.2	10.9
	Blender (sensor-A calib.)	0.0147	0.125	0.846	16.5	11.2
	Blender (domain rand.)	0.0146	0.126	0.830	16.3	10.9
	GAN-25 (sensor B)	0.0107	0.100	0.846	14.6	10.4
	Teacher-label self-training, model only	0.0098	0.100	0.841	14.8	11.6
	Teacher-label self-training	0.0102	0.098	0.840	14.6	11.8
50	Real only	0.0063	0.081	0.860	13.2	9.9
	Blender (sensor-A calib.)	0.0058	0.077	0.858	12.8	9.7
	GAN-50 (sensor B)	0.0045	0.068	0.866	12.1	10.1
100	Real only	**0.0032**	0.058	0.861	**11.0**	**7.5**
	Blender (sensor-A calib.)	0.0033	**0.057**	0.868	11.0	9.3
	Blender (domain rand.)	0.0036	0.062	0.866	11.4	10.6
	GAN-100 (sensor B)	0.0034	0.060	**0.870**	11.2	10.1

TABLE VIII: **Sensor B (GelSlim 4.0).** 1,620 test frames of the same 12 objects. Top: sensor-A models applied zero-shot. Below: models trained on K sensor-B real frames per object, alone, with the sensor-A PBR, with a learned renderer trained on the same K sensor-B pairs, or with self-training on teacher labels (MeshTac without its alignment step, Sec. III; labeler: the GAN-25 model; 50 epochs). Rotation after removing a per-object offset.

K	Training data	Depth MSE	Cont. MAE	IoU	Rot.
10	Real only	0.0317 ±0.0003	0.181 ±0.006	0.780 ±0.001	13.5 ±0.4
10	MeshTac	0.0169 ±0.0001	0.121 ±0.0003	0.855 ±0.0002	10.1 ±0.5
25	Real only	0.0179 ±0.0004	0.127 ±0.001	0.849 ±0.002	12.3 ±0.8
25	MeshTac	0.0076 ±0.0001	0.082 ±0.002	0.912 ±0.001	9.0 ±0.1
50	Real only	0.0089 ±0.0001	0.084 ±0.002	0.900 ±0.001	9.2 ±0.5
50	MeshTac	0.0041 ±0.0001	0.063 ±0.002	0.933 ±0.001	9.8 ±0.05
100	Real only	0.0050 ±0.0001	0.067 ±0.002	0.933 ±0.0001	9.3 ±0.7
100	MeshTac	0.0033 ±0.00002	0.054 ±0.002	0.944 ±0.0003	9.4 ±0.4

TABLE IX: **Sensor B over three seeds** (mesh-registered labels, 50 epochs): mean ± standard deviation of real-only training and renderer-free MeshTac at every budget.

mesh-registered labels and a real-only teacher it does not improve on real data at any budget (Table VII). In both protocols the student inherits the localization of its labeler, which is what alignment corrects.

D. Mesh-Assisted Test-Time Pose Refinement

We reuse the fixed K-shot aligner to refine test-frame poses against their known meshes (Table X). On the validation split rotation error falls from 3.2–9.6° to 0.4–2.8° for real-only models and from 2.5–4.7° to 0.4–2.4° for MeshTac models; from $K = 25$ on the refined error is about 1° whichever model proposes the pose, and translation error falls by 20–50%.

On the second session rotation error falls from 13–20° to 5–11° (real only) and from 10–14° to 5–7° (MeshTac) with a median of 1.5–2.2°: the gap between mean and median indicates some large residual errors.

The aligned mesh also provides a depth estimate: on the validation split it is as accurate as the MeshTac student's output (IoU 0.82 vs. 0.81 at $K = 100$) and lifts a real-only model to the MeshTac level (IoU $0.656 \rightarrow 0.710$ at $K = 25$, MeshTac 0.712), showing that the student can approximate the reconstruction accuracy of mesh-assisted inference.

We also test refinement on held-out objects: trained on the nine seen objects of the leave-object-out protocol and applied to the three held-out ones, whose real contacts were excluded from training, it lowers the rotation error of the simulation-trained models of Table XI from 4.8–6.8° to 0.6–1.0°.

E. Qualitative Results

Fig. 7 compares predictions on held-out frames at $K = 25$. For each of the three displayed objects per sensor, we select the frame with the median contact-MAE improvement. This fixes selection within each object; the examples illustrate local reconstruction differences rather than the full distribution of results. Real-only models blur or break thin ridges and misplace junctions; the MeshTac student recovers their extent and position. On sensor B the real-only model is already close and the gain shows at junctions and at the ends of ridges.

	Validation (session 1)				Session 2			
	Real only		MeshTac		Real only		MeshTac	
K	pred.	+align	pred.	+align	pred.	+align	pred.	+align
Rotation error (deg)								
10	9.6	2.8	4.7	2.4	20.2	11.4	14.2	7.0
25	5.6	1.2	2.8	0.9	16.2	5.5	10.9	5.2
50	4.3	1.2	2.7	1.0	13.8	4.8	11.3	7.4
100	3.2	0.4	2.5	0.4	12.9	4.9	10.1	4.9
Translation error (mm)								
10	1.69	1.01	1.13	0.89	2.25	1.57	1.97	1.55
25	1.21	0.80	0.91	0.71	2.03	1.27	1.71	1.25
50	0.91	0.52	0.74	0.47	2.02	1.27	1.73	1.29
100	0.72	0.34	0.70	0.36	1.84	1.25	1.76	1.25

TABLE X: **Test-time alignment.** Rotation error (deg) of the predicted pose and after six aligner iterations against the object mesh. No retraining.

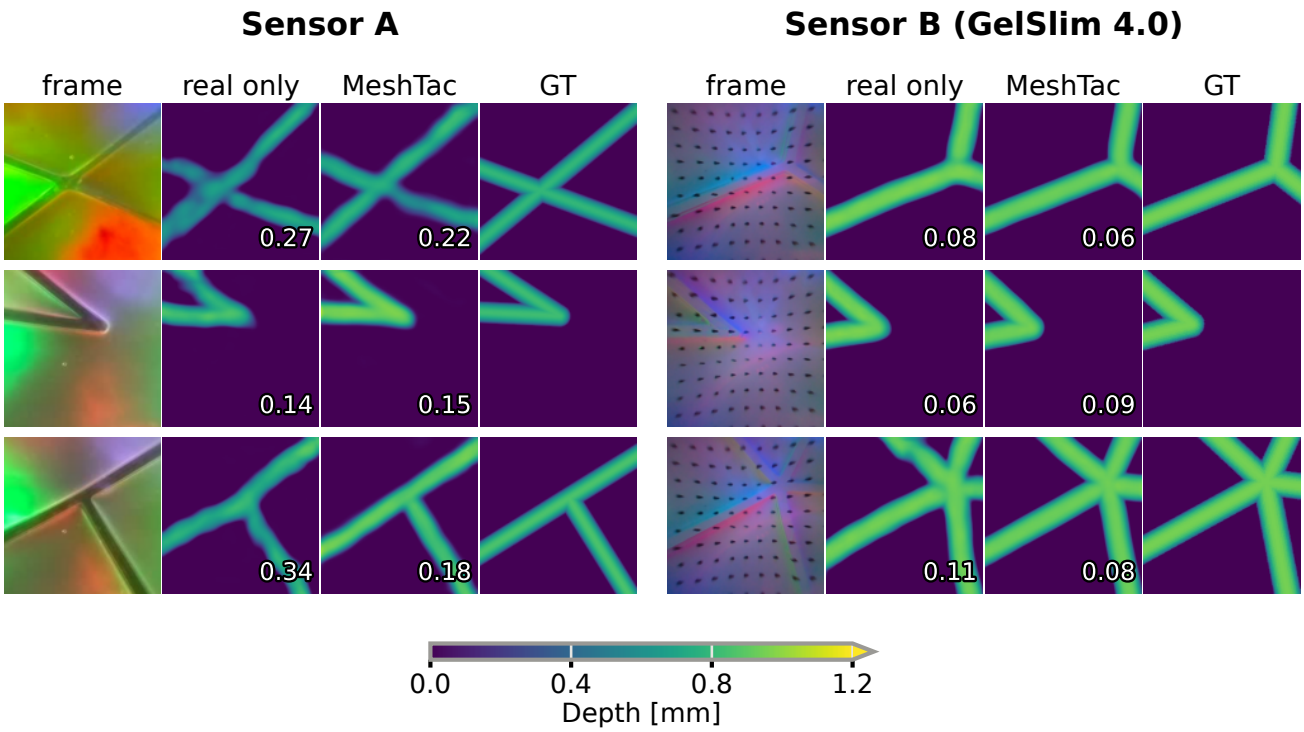


Fig. 7: **Depth predictions.** $K = 25$, held-out frames of three selected objects per sensor. Each example is the frame with the median MeshTac contact MAE for its object, chosen without reference to the baseline. Numbers are contact MAE (mm); depth maps share a color scale.

F. Renderer Baselines for Unseen Objects

Three objects are removed from all real training while their meshes remain available to simulation (Table XI).

Pose. Without training data for the three objects, the pose head has a large rotation error ($64.7°$). Simulated contacts of the three meshes bring it to $6.8°$ with PBR—with no-contact illumination calibration and manually tuned contact parameters—and to $5.7°$ with the strict learned renderer; the learned renderer (full) gives a smaller further reduction ($4.6°$), and the geometry control (same renderer, meshes removed) stays at $50.6°$. The geometry control supports mesh coverage as a major source of the pose improvement. Replacing the calibrated renderer with domain-randomized sensor parameters (random LED colors and strengths, gel optics randomized per session, geometry pinned) gives $8.9°$, between the calibrated ($6.8°$) and single-rotation ($10.3°$) rows; a consistent rendered appearance may make geometric cues easier to recover.

Rotation coverage. We vary the simulated rotation coverage to examine its contribution to pose transfer. Restricting simulation to one real rotation per object leaves $10.3°$, two rotations $7.5°$, all twelve $6.8°$ (Blender rows of Table XI); rendering the learned renderer's inputs after rotating the depth map (rotate-then-render) gives the best leakage-free pose, $4.4°$. These results support covering the expected rotation range when generating synthetic contacts.

Depth and normals. Contact MAE on the unseen objects is 0.36 mm for real-only training, three times the seen-object value; the plain renderers reduce it by at most 7% and the geometry control by almost as much (0.340), suggesting that these small dense gains may reflect regularization as well as object coverage. The domain-randomized Blender renderer improves contact MAE to 0.342 and IoU to 0.601, matching the calibrated renderer (0.346, 0.592), suggesting that appearance diversity from randomized sensor parameters acts as regularization even without calibration data. A multi-sensor GAN trained on 100 simulated sensor designs does not improve over single-sensor baselines (0.358, 0.579); sensor diversity does not replace target-sensor pairs in this experiment. The strongest improvement among renderers that exclude held-out real pairs comes from the background-conditioned GAN-LOO: depth MSE -13%, contact MAE 0.323, IoU 0.633, for one background frame per session; it remains far from the learned renderer (full) (-30%), which has seen real contacts of the objects. Normal error follows contact MAE. Peak-depth error is 0.05–0.08 mm for every row: the models estimate *how deep* correctly and err in *where* the contact lies.

Object classification. We add a 12-class classification head (LayerNorm + MLP on the class and mean patch tokens) and train with cross-entropy. With real data of the nine objects, the model classifies them at 94% validation accuracy, but the three held-out objects score 0%—even when simulation provides their contact patterns through either the calibrated or learned renderer. Synthetic geometry supports pose transfer in this setting, but the same data does not establish transfer for object classification. Appearance mismatch remains a possible explanation for the difference.

Method	Depth MSE↓	Contact MAE↓	Contact IoU↑	Normal (deg)↓	Rot. (deg)↓	Trans. (mm)↓
Real only (9 objects)	0.0665	0.356	0.582	30.4	64.8	4.04
Blender	0.0657	0.346	0.592	30.3	6.9	**1.31**
Blender (domain rand.)	0.0635	0.342	0.601	30.4	8.9	1.55
Blender, bg-norm.	0.0672	0.348	0.589	30.3	5.1	1.35
Blender, 1 rotation / object	0.0645	0.354	0.585	30.4	10.2	1.61
Blender, 2 rotations / object	0.0670	0.345	0.590	30.8	7.6	1.71
GAN-LOO	0.0631	0.331	0.618	29.9	5.6	1.44
w/o held-out meshes (control)	0.0647	0.340	0.604	30.0	50.5	4.57
Bg-cond. GAN-LOO	**0.0581**	**0.323**	**0.633**	**29.1**	4.8	1.54
GAN-LOO, rotate-then-render	0.0625	0.345	0.606	29.6	**4.5**	1.37
Multi-sensor GAN-LOO	0.0695	0.358	0.579	30.9	8.9	1.60
Blender ⊕ GAN-LOO	0.0661	0.349	0.601	30.7	5.5	1.40
Refiner (0 labels)	0.0673	0.349	0.590	30.5	7.7	1.86
Refiner on domain rand. (0 labels, no calib.)	0.0691	0.353	0.583	30.8	7.1	1.86
GAN-full (full real set)	*0.0467*	*0.277*	*0.676*	*25.8*	*4.5*	*1.40*

TABLE XI: **Leave-object-out.** Errors on 1,244 real frames of three objects with no real training data. Rows add simulated contacts of all 12 meshes to the real frames of the nine other objects, unless noted. The multi-sensor row is a 50-epoch run. Normal error is inside the contact region. Bold: best leakage-free.

G. Renderer Baselines Across Label Budgets

Tables I and II, and Fig. 4 vary the real budget of the renderers. Three regimes appear.

Zero-shot. Simulation alone transfers poorly for dense geometry (contact MAE 0.41 mm vs. 0.11 with the full real set); the learned renderer (full) halves this but has seen 4,197 real pairs.

Few-shot ($K \leq 25$). Neither budget-matched paired renderer improves both contact MAE and IoU. At $K = 25$ GAN-25 lowers depth MSE by 3% (0.0536 ± 0.0013 vs. 0.0551 ± 0.0010 over seeds) but not contact MAE or IoU; PBR does the same. With $K \leq 25$ the real training loss reaches 10^{-4} within ten epochs while validation error is flat from epoch five: the separation between training and validation error is consistent with overfitting, which the evaluated synthetic data does not resolve. Simulation does help pose at $K = 10$ ($9.6° \rightarrow 5.2$–$5.9°$), as in Sec. V-F.

Moderate budget ($K \geq 50$). The learned renderer now helps: at $K = 50$ depth MSE $0.0482 \rightarrow 0.0434$ (-10%), contact MAE $0.283 \rightarrow 0.260$, IoU $0.69 \rightarrow 0.72$; at $K = 100$ depth MSE $0.0369 \rightarrow 0.0311$ (-16%; $0.0366 \rightarrow 0.0299$ over seeds, -18%), contact MAE $0.235 \rightarrow 0.210$, IoU $0.74 \rightarrow 0.77$, and normal error improves in step. PBR gives -1 to -5% MSE. The learned renderer thus needs ≈ 50 labeled contacts per object before its synthetic data improves contact reconstruction over training on the same real frames alone, and at every $K \geq 50$ simulation *worsens* rotation by 0.2–1.3°.

H. Label Leakage, Renderer Fidelity and Negative Results

Renderer supervision. Across Tables XI and I the learned renderer (full) reports depth MSE reductions of 24–26% at the reported K-shot budgets and 30% on unseen objects; the strict numbers are 0–3% for $K \leq 25$, 10–16% at $K \geq 50$ and 5–13% on unseen objects. The gap is largest when the downstream model has few or no real contacts of the target object, while GAN-full receives additional labeled contacts. In LOO it also receives contacts from the held-out objects. Validation frames are excluded from GAN-full training; the protocol violation concerns the training-label budget or held-out object access.

A fidelity proxy. Rendering the depth maps of the real validation pairs and comparing with the real images gives an image error per renderer. Fig. 8 plots the contact-region L1 against the downstream depth MSE for every learned-renderer variant: lower image error is associated with lower downstream MSE in this collection. Over the strict paired renderers, pixel metrics have the strongest rank association with downstream error (Spearman $|\rho|$: contact L1 0.95 over the 14 renderers of Fig. 8, 0.97 over the nine original ones; PSNR 0.98, full-image L1 0.95), perceptual metrics worse (SSIM 0.87, LPIPS 0.87, contact-cropped 0.90); adding the three learned renderer (full)s, whose fidelity is identical while their downstream error varies with K, lowers every correlation to 0.75–0.82. These pooled correlations mix budget and renderer changes and do not isolate the causal effect of image fidelity.

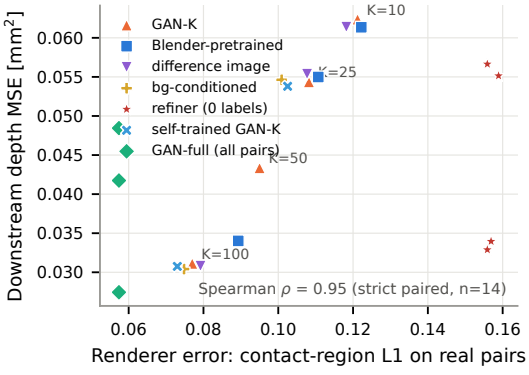


Fig. 8: **Renderer fidelity and downstream error.** Contact-region L1 between rendered and real images on held-out real (depth, image) pairs versus depth MSE of the downstream model trained with that renderer's data, for every learned-renderer variant and budget.

The unpaired refiner uses a different fidelity comparison: it is scored against the gel-warped physics render rather than its own input, so its contact L1 (0.16) is not comparable (Sec. V-B). The image metric was used to screen the variants below before downstream training.

Which factor limits few-shot fidelity?

Table XII examines physical conditioning, illumination, training choices and appearance randomization as possible sources of the few-shot gap. *(i) Missing physical prior.* Pretraining the GAN on 30k Blender pairs, conditioning it on the Blender image of the same contact, predicting a residual on top of that image, or conditioning on Blender image and background all leave contact-region fidelity unchanged or worse at $K \in \{10, 25\}$ (PBR's random gel-warp of ± 3 pixels means the conditioning image is not pixel-aligned with the target, which may limit this family).

(ii) Session illumination drift. Session backgrounds differ by 0.05–0.10 L1, more than the best renderer's total error (0.03); normalizing frames to a canonical background or predicting difference images improves full-image L1 by 13–21%

	$K = 25$		Unseen objects		
Variant	Fidelity ↓	Contact MAE ↓	Fidelity ↓	Contact MAE ↓	Rot. (deg) ↓
Real only	–	0.312	–	0.356	64.8
GAN-K / GAN-LOO	0.108	0.321	0.068	0.331	5.6
Blender-pretrained	0.111	0.335	0.084	0.356	8.5
difference image	0.108	0.339	0.066	0.346	6.2
background-conditioned	**0.101**	0.330	**0.062**	**0.323**	**4.8**
physics-conditioned	0.114	–	–	–	–
physics residual	0.119	–	–	–	–
physics + background	0.104	–	–	–	–
self-trained (+ pseudo pairs)	0.102	0.326	–	–	–
Refiner, unpaired (0 labels)	0.159	0.297	0.159	0.349	7.7
+ self-regularization	–	–	0.156	0.360	7.4
on domain-rand. render	–	–	0.159	0.353	7.1
Blender (calibrated)	–	0.329	0.179	0.346	6.9
Blender (domain rand.)	–	0.320	–	0.342	8.9
Blender, bg-norm.	–	0.338	–	0.348	5.1
Real only, mask head + grad. loss	–	0.309	–	–	–
GAN-K, mask head + grad. loss	–	0.302	–	–	–
Blender pretrain → real fine-tune	–	**0.294**	–	–	–
+ L2-SP	–	0.307	–	–	–

TABLE XII: **Appearance-side and training-side variants** at $K = 25$ (real validation split) and leave-object-out (unseen objects). Fidelity is the contact-region L1 of Fig. 8; physics-guided variants were stopped at the fidelity stage. Only the background-conditioned renderer improves unseen-object depth.

but leaves contact-region L1 and contact MAE unchanged (consistent with some encoder tolerance to background illumination). Conditioning the renderer on the background does help on unseen objects (Sec. V-F) where the background frame supplies session-specific illumination information.

(iii) Training-side changes. A contact-mask head with a gradient-matching loss, Blender pretraining followed by real fine-tuning, and fine-tuning with an L2-SP anchor change $K = 25$ contact MAE by at most ± 0.02 mm.

(iv) Domain randomization. Randomizing the sensor's LED colors, strengths and gel optics per session while keeping the geometry stage fixed (Table XII, "domain rand.") matches the calibrated renderer's contact MAE at $K = 25$ (0.320 vs. 0.329) and on unseen objects (0.342 vs. 0.346). On sensor B at $K = 25$ the match is even closer (0.126 vs. 0.125). The calibrated renderer's advantage on pose (6.8° vs. 8.9° on unseen objects) is consistent with a less noisy silhouette. Domain randomization is thus a useful baseline without appearance calibration when calibration data is not available, but it cannot close the gap to the learned renderer.

(v) Contact shading. The appearance *inside* the contact remains a plausible limitation (Fig. 3): strict renderers separate from real-only training only after enough contacts, additional paired labels improve renderer performance, and the tested synthetic-data configurations do not reliably transfer object classification. These observations implicate appearance mismatch, but do not isolate contact shading as the sole cause.

Unpaired refiner (0 labels). The contact-preserving refiner of Sec. IV-D translates the physical render using only unlabeled frames. On unseen objects (Table XI) it does not improve on the calibrated render (contact MAE 0.349 vs. 0.346; on the domain-randomized render 0.353), and its self-regularized variant is slightly worse (0.360): one possible explanation is that adversarial training rewards plausible shading without sufficiently constraining the geometry needed for downstream prediction.

In the K-shot protocol, refining with the same $K = 25$ frames used as unlabeled images improves over the paired-renderer baselines at that budget (contact MAE 0.297, IoU 0.672, normal 27.1°; Table I), a 5% contact-MAE gain where no paired renderer helps, while using all of U (0.309) or $K = 100$ (0.224 vs. real-only 0.235, GAN-100 0.210) adds little. The refiner's contact-region L1 (0.156–0.159, against the ± 3 px-warped physics render whose identity score is 0.179) is not comparable to the paired renderers' and is excluded from Fig. 8.

Mixing the two renderers. Alternating physical and learned frames (Blender $\oplus$ GAN) yields the average of the two on dense error and the better of the two on pose (5.6° on unseen objects, equal to GAN-LOO; 9.8° across sessions, the lowest of all renderers except background normalization). This suggests a task-dependent effect of mixing, rather than a consistent gain across outputs.

VI. Discussion, Limitations and Conclusion

Practical recipe. For a new tactile sensor and known object meshes, first select a small set of registered contacts spanning

the intended contact states. Reserve a separate labeled validation set and report its size alongside the training budget. Train a teacher and a residual aligner on the available training labels; use the aligner to assign mesh-derived depth, normal and pose targets to additional real images. Train a student on the labeled and aligned frames. For the evaluated objects, one alignment round without confidence filtering captures the observed gains; further rounds offer little improvement.

A practical starting point is MeshTac without any renderer: it gives the within-session gains and avoids appearance-model training. When the deployment session differs from the training session, our results support adding synthetic data from a renderer retrained on the labeled and aligned real pairs. At $K = 100$ on sensor A, this lowers second-session contact MAE from 0.288 to 0.252 mm. If real contacts of a new object are unavailable but its mesh is known, synthetic pose coverage is also useful: the held-out-object study reduces rotation error from $64.7°$ to 5–7°. Optional test-time alignment can further refine pose when object identity and mesh are available, but its larger residuals across sessions should be considered for the intended application. These recommendations concern the evaluated ridge-pattern contacts; they do not establish a universal label threshold for other sensors or geometries.

Relation to renderer calibration. One could view simulation as needing only to be *task-sufficient*: it must preserve the geometry-conditioned appearance cues the downstream model relies on, not the physical parameters of the sensor. Our results agree that the geometric cues are what transfers, but on our data no leakage-free renderer, calibrated or learned from few pairs, reproduced the contact appearance well enough to improve dense prediction; aligning the geometry directly to real frames obtained those cues where appearance calibration did not.

Reporting. Report the real data used by every component, including the renderer, teacher, aligner and checkpoint-selection procedure. Distinguish labeled training pairs, unlabeled images, validation labels and images inspected during manual calibration. A renderer fitted to all available training pairs can exceed a downstream K-shot budget without seeing a validation image. In leave-object-out evaluation, access to real images of the held-out objects is a separate protocol violation. Both should be distinguished from access to the held-out meshes, which is permitted in our geometry-transfer study.

Under this matched budget, the learned renderer (full)'s 24–26% depth-MSE reduction over K-shot real-only training is a result at a larger label budget, not a gain from simulation at a cost of K labels. Report metrics that distinguish depth magnitude from contact localization, since all-pixel MSE can improve while contact MAE or IoU worsens. Also identify training duration, seed aggregation, checkpoint selection and test-time mesh use. Contact-region rendering error can help screen appearance models, but its observed correlation with downstream error pools multiple budgets and renderer variants; it should complement downstream evaluation rather than replace it.

Limitations. MeshTac targets known-object perception. Object identity is used by the evaluated predictor, and a mesh is additionally required during pseudo-labeling and optional test-time refinement. The student itself does not render the mesh at standard inference. Because GT and pseudo-labels are mesh projections, the experiments assess recovery of registered object geometry rather than physical elastomer deformation; sensor-B targets are additionally re-registered from public labels. The 12 ridge-pattern objects provide controlled tests of contact localization and pose, but do not establish generalization to curved or textured contacts or unknown objects. Validation labels are also needed for checkpoint selection. Alignment can fail, particularly across sessions, and repeated ridge patterns leave some translations ambiguous. All contacts are quasi-static presses; shear, slip, frictional hysteresis and gel aging are not modeled or studied. Sensor-A object identity and session appearance are confounded because each object has one training capture session. PBR uses a geometric approximation to gel deformation, and its manual contact-parameter tuning belongs in the data-access account. Finally, we evaluate perception rather than its effect on a manipulation task.

Conclusion. MeshTac uses known object geometry to turn unlabeled real contacts into supervision for tactile perception. Refining the contact state improves both the mesh-derived labels and the student trained on them. That these gains require no synthetic images indicates that mesh-pose accuracy is a key bottleneck in this setting. Synthetic data contributes under session changes, and the same aligner supports optional pose refinement at inference. The accompanying renderer study shows why the real training labels used by every component must be counted when assessing label efficiency. Future work will examine alignment under less constrained contact geometry and its use in downstream control.

REFERENCES

[1] W. Yuan, S. Dong, and E. H. Adelson, "Gelsight: High-resolution robot tactile sensors for estimating geometry and force," *Sensors*, vol. 17, no. 12, 2017.

[2] E. Donlon, S. Dong, M. Liu, J. Li, E. Adelson, and A. Rodriguez, "Gelslim: A high-resolution, compact, robust, and calibrated tactile-sensing finger," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2018.

[3] M. Lambeta, P.-W. Chou, S. Tian, B. Yang, B. Maloon, V. R. Most, D. Stroud, R. Santos, A. Byagowi, G. Kammerer, D. Jayaraman, and R. Calandra, "Digit: A novel design for a low-cost compact high-resolution tactile sensor with application to in-hand manipulation," *IEEE Robotics and Automation Letters*, vol. 5, no. 3, pp. 3838–3845, 2020.

[4] S. Wang, M. Lambeta, P.-W. Chou, and R. Calandra, "Tacto: A fast, flexible, and open-source simulator for high-resolution vision-based tactile sensors," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 3930–3937, 2022.

[5] Z. Si and W. Yuan, "Taxim: An example-based simulation model for gelsight tactile sensors," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 2361–2368, 2022.

[6] Z. Chen, S. Zhang, S. Luo, F. Sun, and B. Fang, "Tacchi: A pluggable and low computational cost elastomer deformation simulator for optical tactile sensors," *IEEE Robotics and Automation Letters*, vol. 8, no. 3, pp. 1239–1246, 2023.

[7] A. Agarwal, T. Man, and W. Yuan, "Simulation of vision-based tactile sensors using physics based rendering," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 1–7.

[8] J. Xu, S. Kim, T. Chen, A. R. Garcia, P. Agrawal, W. Matusik, and S. Sueda, "Efficient tactile simulation with differentiability for robotic manipulation," in *Proceedings of The 6th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 205. PMLR, 2023, pp. 1488–1498. [Online]. Available: https://proceedings.mlr.press/v205/xu23b.html

[9] I. Akinola, J. Xu, J. Carius, D. Fox, and Y. Narang, "Tacsl: A library for visuotactile sensor simulation and learning," *IEEE Transactions on Robotics*, 2025.

[10] A. Church, J. Lloyd, R. Hadsell, and N. F. Lepora, "Tactile sim-to-real policy transfer via real-to-sim image translation," in *Proc. Conf. Robot Learning (CoRL)*, 2022, pp. 1645–1654.

[11] W. Chen, Y. Xu, Z. Chen, P. Zeng, R. Dang, R. Chen, and J. Xu, "Bidirectional sim-to-real transfer for gelsight tactile sensors with cyclegan," *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 6187–6194, 2022.

[12] S. Zhong, A. Albini, P. Maiolino, and I. Posner, "Tactgen: Tactile sensory data generation via zero-shot sim-to-real transfer," *IEEE Transactions on Robotics*, vol. 41, pp. 1316–1328, 2025.

[13] A. Shrivastava, T. Pfister, O. Tuzel, J. Susskind, W. Wang, and R. Webb, "Learning from simulated and unsupervised images through adversarial training," in *CVPR*, 2017.

[14] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros, "Unpaired image-to-image translation using cycle-consistent adversarial networks," in *ICCV*, 2017.

[15] T. Park, A. A. Efros, R. Zhang, and J.-Y. Zhu, "Contrastive learning for unpaired image-to-image translation," in *ECCV*, 2020.

[16] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2017.

[17] H. Gupta, Y. Mo, S. Jin, and W. Yuan, "Sensor-invariant tactile representation," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2025.

[18] J. Zhao, Y. Ma, L. Wang, and E. Adelson, "Transferable tactile transformers for representation learning across diverse sensors and tasks," in *Proc. Conf. Robot Learning (CoRL)*, 2024.

[19] C. Higuera, A. Sharma, C. K. Bodduluri, T. Fan, P. Lancaster, M. Kalakrishnan, M. Kaess, B. Boots, M. Lambeta, T. Wu, and M. Mukadam, "Sparsh: Self-supervised touch representations for vision-based tactile sensing," in *Proc. Conf. Robot Learning (CoRL)*, 2024.

[20] D.-H. Lee, "Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks," in *ICML Workshop on Challenges in Representation Learning*, 2013.

[21] Q. Xie, M.-T. Luong, E. Hovy, and Q. V. Le, "Self-training with noisy student improves imagenet classification," in *CVPR*, 2020.

[22] E. Arazo, D. Ortego, P. Albert, N. E. O'Connor, and K. McGuinness, "Pseudo-labeling and confirmation bias in deep semi-supervised learning," in *IJCNN*, 2020.

[23] Y. Li, G. Wang, X. Ji, Y. Xiang, and D. Fox, "DeepIM: Deep iterative matching for 6D pose estimation," in *ECCV*, 2018.

[24] G. Wang, F. Manhardt, J. Shao, X. Ji, N. Navab, and F. Tombari, "Self6D: Self-supervised monocular 6D object pose estimation," in *ECCV*, 2020.

[25] A. Sipos, W. van den Bogert, and N. Fazeli, "GelSlim 4.0: Focusing on touch and reproducibility," *arXiv preprint arXiv:2409.19770*, 2024.

[26] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, "Image-to-image translation with conditional adversarial networks," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 1125–1134.

[27] R. Ranftl, A. Bochkovskiy, and V. Koltun, "Vision transformers for dense prediction," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2021, pp. 12 179–12 188.

[28] A. Kendall, Y. Gal, and R. Cipolla, "Multi-task learning using uncertainty to weigh losses for scene geometry and semantics," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 7482–7491.